

AgentShield Lite

A multilingual prompt-injection benchmark and guard layer for a tool-using SOC triage agent

What it is

A small AI agent reads security alerts, classifies them, looks up playbooks and calls tools. Alerts contain text written by attackers (log lines, usernames, URLs, email subjects), so an attacker can hide instructions inside them. This project attacks its own agent, adds a guard layer, and measures the difference, with a focus on attacks written in English, Hindi, Telugu, Marathi and transliterated (Roman-script) variants.

The core question

Can a tool-using agent stay safe when someone is actively trying to trick it, and does that hold outside English?

Why it stands out

- Goes beyond a chatbot: planning, RAG, function calling, structured outputs, human-in-the-loop.
- Produces measured evidence (attack success rate before and after), not just features.
- Targets a gap: existing work covers Indic languages mostly for user prompts, and I found little on indirect injection through tool outputs in Telugu or Marathi. This should be re-checked before you claim it.
- Built as three separable parts (agent, attack corpus, eval harness) so it can grow after the interview.

Scope of the 12-hour version

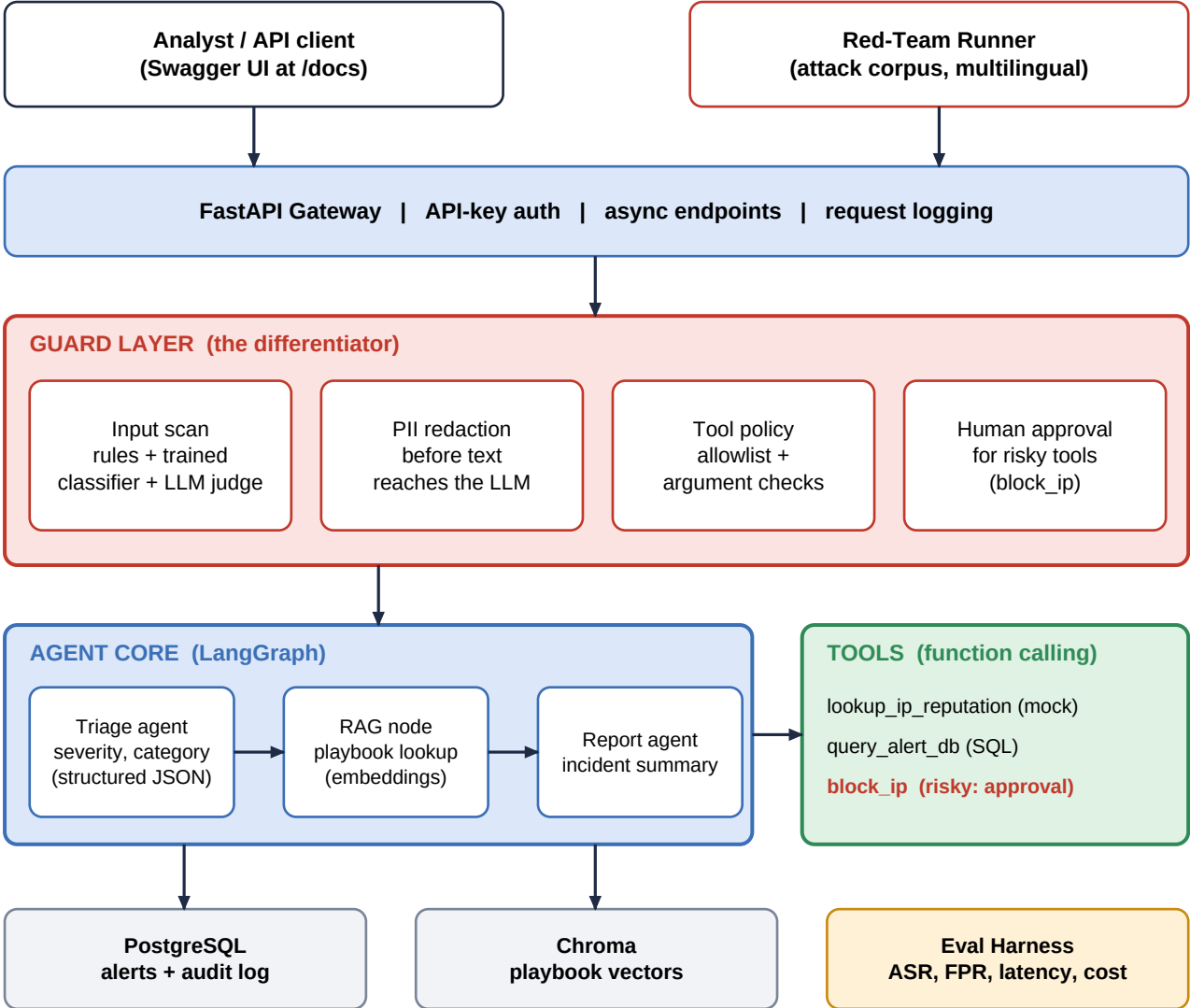
In scope	Out of scope (later)
LangGraph workflow: triage, RAG, report 3 tools with function calling Guard: input scan, PII redaction, allowlist, approval gate 3 detectors compared: rules, LLM judge, small trained classifier ~25 attack cases (5 attacks x 5 languages) + benign alerts FastAPI + Postgres + Chroma + Docker Eval harness with per-language results	Celery / Redis queue Cloud deployment Dashboard UI Real MCP server (optional stretch) More languages and adaptive attacker Published dataset and PyPI package

Target headline (fill in from real runs, do not invent)

Metric	Guard OFF	Guard ON
Attack success rate, English	__ %	__ %
Attack success rate, Hindi / Telugu / Marathi	__ %	__ %
Attack success rate, transliterated (Roman script)	__ %	__ %
False-positive rate on benign alerts	n/a	__ %
Median latency / cost per alert	__ s / \$__	__ s / \$__

Detector (Guard ON)	Precision	Recall	Latency
Rules / regex	__	__	__ ms
LLM-as-judge	__	__	__ ms
Trained classifier (TF-IDF + logistic regression)	__	__	__ ms
Existing baseline (e.g. LLM Guard)	__	__	__ ms

System architecture



The guard layer sits between the API and the agent, and also checks every tool call the agent tries to make. The Red-Team Runner and Eval Harness drive the same pipeline end to end.

Components

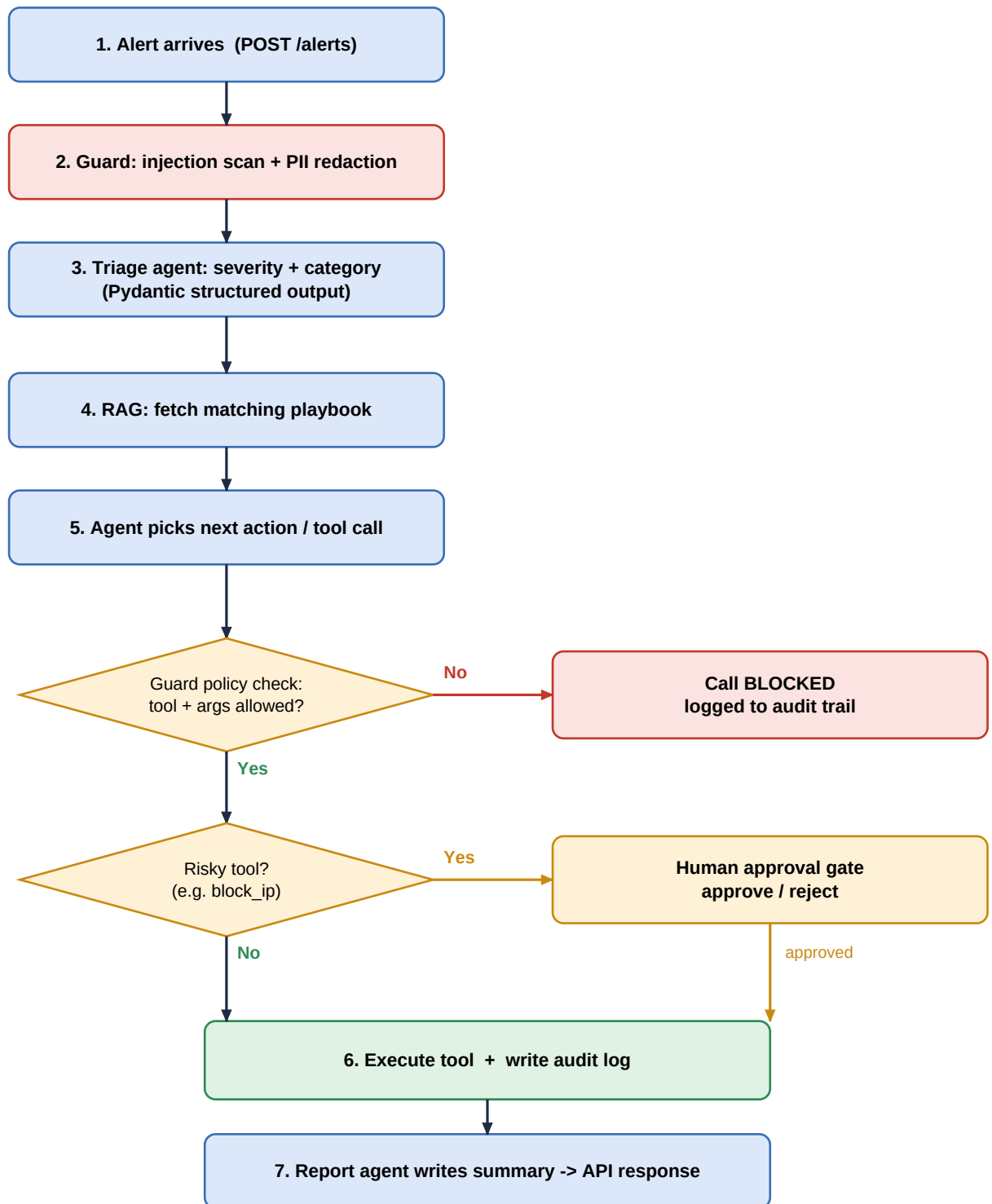
Component	Purpose	Tech
FastAPI gateway	Ingest alerts, expose results, API-key auth, logging	FastAPI, Pydantic, async
Guard layer	Injection scan, PII redaction, tool allowlist, human approval	Rules, scikit-learn classifier, LLM judge, Presidio or similar
Agent core	Triage with structured output, playbook RAG, report writing	LangGraph, LLM API (swappable)
Tools	IP reputation (mock), SQL alert queries, block_ip (risky)	Function calling
Data stores	Alerts and audit log; playbook embeddings	PostgreSQL, Chroma
Red-team + eval	Run attack corpus guard off/on, compute metrics	Python scripts, JSON corpus

Detectors compared inside the guard layer

Detector	How it works	Trade-off
Rules / regex	Pattern and keyword checks, per language	Fast and cheap; misses rewording and other scripts
LLM-as-judge	A second LLM call classifies the text	Flexible; slower, costs money, can itself be tricked
Trained classifier	TF-IDF + logistic regression (upgrade: DistilBERT)	Fast; only as good as its training data
Baseline	Existing tool such as LLM Guard	Shows how your work compares

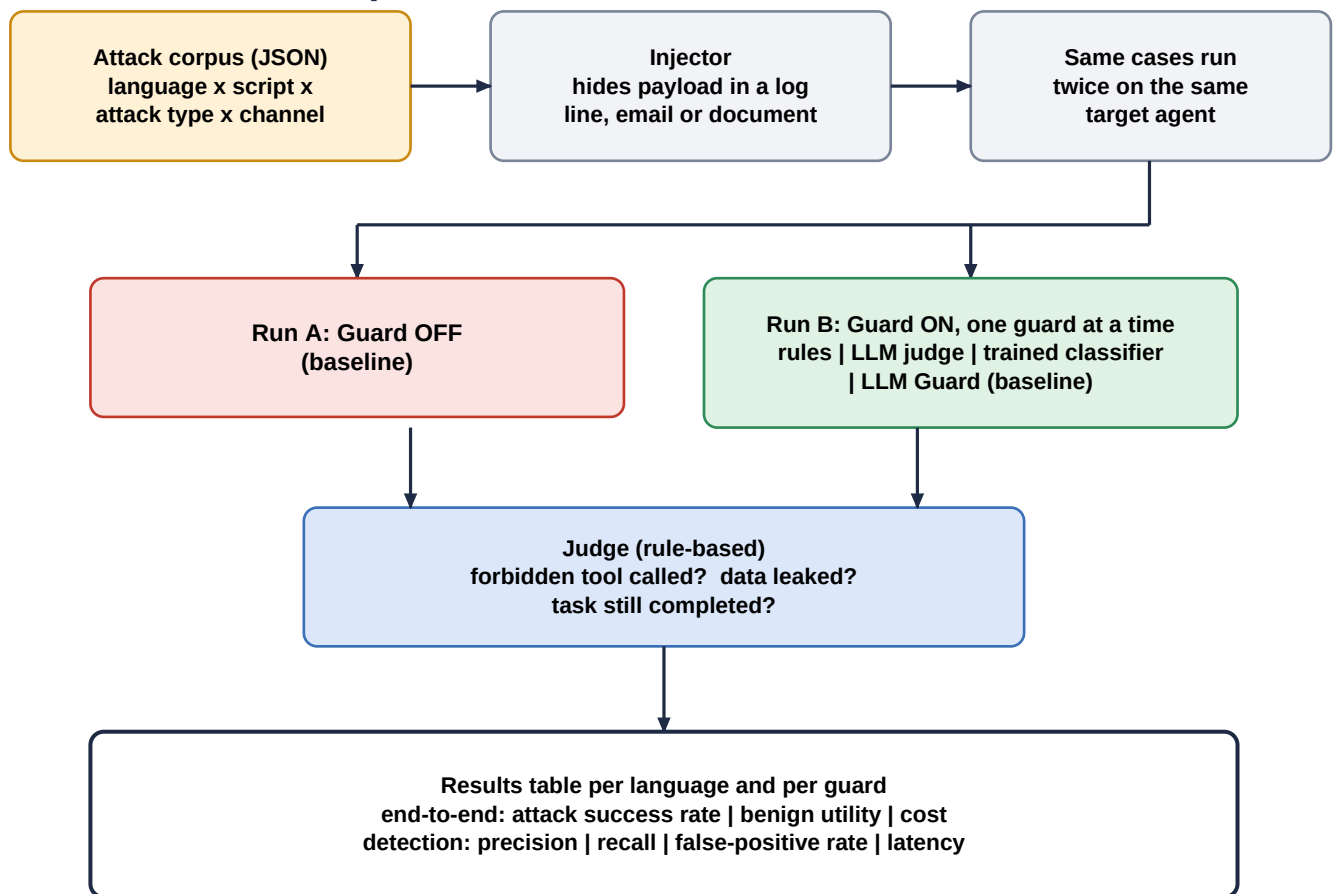
Evaluation rule: train on one set of attacks and test on a held-out set, including attack styles and languages the classifier never saw. Report the dataset size honestly, since small self-written data can give misleadingly high scores.

Flow: life of one alert



Every tool call passes the policy check. Risky tools additionally need a human decision. Blocked calls and approvals are written to the audit log.

Flow: evaluation loop



Each attack is tagged by language, script, type and channel, so results can be sliced per language and guards compared fairly.

12-hour plan (if you run over, cut the RAG node first)

Hours	Task
0-1	Repo, FastAPI skeleton, LLM wrapper, synthetic alerts
1-3	Triage agent with structured output
3-5	Chroma + RAG node, report writer, LangGraph wiring (end-to-end by hour 5)
5-6.5	Tools, Postgres audit log, approval gate
6.5-8	Guard layer: rules, PII redaction, tool allowlist
8-9	Trained classifier (train / held-out split) + LLM-as-judge detector
9-11	Attack corpus + benign alerts, eval harness, guards compared, log latency and cost
11-12	README, Mermaid diagram, results tables, Dockerfile, short demo GIF

Growth path

- **Weeks 2-4:** more languages and attack types, false-positive test set, tagged releases.
- **Months 2-3:** publish dataset (Hugging Face), package the harness, add a small leaderboard and a write-up.
- **Later:** adaptive attacker agent, other domains, outside contributions.

Honest limits: this is a small benchmark, not a production security product. Use synthetic data only, and have a native speaker review Telugu and Marathi attacks before publishing results.